

Module 1

Introduction To C: The C Character Set, Identifiers And Keywords, Data Types, Constants, Variables, Declarations, Expressions, Statements. Data Input And Output: Single Character Input, Single Character Output, Scanf, Printf, Operators And Expressions: Arithmetic Operators, Unary Operator, Relational And Logical Operator, Assignment Operators, The Conditional Operator, Type Conversion, Typedef Introduction To Pre-processor Directives.

History of C

History and Development of C

1. Development of BCPL

- In **1967**, **Martin Richards** developed a programming language called **BCPL**.
- BCPL stands for **Basic Combined Programming Language**.
- It was mainly developed for **writing system software**.

2. Development of B

- In **1970**, **Ken Thompson** developed a new language called **B**.
- B used many features of **BCPL**.
- B was used to develop the early versions of the **UNIX operating system** at **Bell Laboratories**.
- Both **BCPL** and **B** were **typeless system programming languages**.

3. Development of C

- In **1972**, **Dennis Ritchie** developed the **C programming language** at **Bell Laboratories**.
- C evolved from **ALGOL, BCPL, and B**.
- C included many concepts from these languages.
- It introduced **data types** and several other powerful features.
- C was developed along with the **UNIX operating system**.
- Therefore, C is strongly associated with UNIX.
- The UNIX operating system was coded almost entirely in **C**.

4. Popularity of C

- Initially, C was mainly used in **academic environments**.
- Later, many **C compilers** became commercially available.
- The increasing popularity of **UNIX** also helped C become widely used.

- C became popular among **computer professionals**.
- Today, C runs on many different **operating systems and hardware platforms**.

5. Traditional C

- During the **1970s**, C evolved into a version known as **Traditional C**.
- The language became more popular after the publication of the book "**The C Programming Language**".
- The book was written by **Brian Kernighan and Dennis Ritchie** in **1978**.
- Because of this book, the language became known as **K&R C**.

6. Need for Standardization

- The rapid growth of C resulted in many different versions of the language.
- These versions were similar but sometimes **incompatible** with each other.
- This created problems for **system developers**.
- To maintain a common standard, the **American National Standards Institute (ANSI)** appointed a technical committee in **1983**.

7. ANSI C / C89

- The committee approved a standard version of C in **December 1989**.
- This version is known as **ANSI C**.
- It was later approved by the **International Organization for Standardization (ISO)** in **1990**.
- This version is also called **C89**.

8. Development of C++

- **C++** was developed based on the C language.
- It added several new features to C.
- C++ became a powerful **object-oriented programming language**.
- It also became more versatile than the original C language.

9. Development of Java

- During the same period, **Sun Microsystems** developed the **Java programming language**.
- Java was influenced by **C and C++**.
- It became a popular programming language for developing different types of applications.

10. Development of C99

- Programming languages continue to improve by adding new features.
- C also continued to develop.
- The C standardization committee introduced a new standard in **1999**.
- This version is known as **C99**.
- C99 included several new and improved features.

11. C Today

- C99 is an improved version of the C language.
- However, many commonly available **C compilers do not support all C99 features**.
- Therefore, some advanced C99 features may not work on all compilers.
- C continues to be an important programming language used on different **operating systems and hardware platforms**.

C Language Evolution – Short Flow

ALGOL → BCPL → B → C → ANSI C (C89) → C99

Important Names:

- **Martin Richards** → BCPL
- **Ken Thompson** → B
- **Dennis Ritchie** → C
- **Brian Kernighan & Dennis Ritchie** → *The C Programming Language* book

Features and Advantages of C Language

The increasing popularity of C is due to its many useful features and advantages.

1. Robust Language

- C is a **robust and powerful programming language**.
- It has a rich set of **built-in functions and operators**.
- It can be used to develop both simple and complex programs.

2. Suitable for System and Business Applications

- C combines the features of a **high-level language** with some capabilities of **assembly language**.
- Therefore, it is suitable for developing:
 - **System software**
 - **Operating systems**
 - **Business applications**

- Many C compilers themselves are written using C.

3. Fast and Efficient

- Programs written in C are generally **fast and efficient**.
- This is because C provides:
 - Different **data types**
 - Powerful **operators**
- C programs are usually faster than programs written using interpreted languages such as BASIC.

4. Simple and Small Language

- ANSI C has a relatively **small set of keywords**.
- Its power comes mainly from its **built-in and library functions**.
- Many standard functions are available to make programming easier.

5. Portability

- C is a **highly portable language**.
- A C program written for one computer can usually be moved to another computer with **little or no modification**.
- This is useful when using different **computers or operating systems**.

6. Supports Structured Programming

- C is well suited for **structured programming**.
- A large problem can be divided into smaller **functions or modules**.
- These modules can be combined to form a complete program.
- This makes the program easier to:
 - **Debug**
 - **Test**
 - **Maintain**

7. Extensibility

- C is an **extensible language**.
- A C program is mainly a collection of **functions**.
- Programmers can create their own functions and add them to their programs.
- The availability of a large number of library and user-defined functions makes programming easier.

“Because of its **speed, efficiency, portability, structured programming features, powerful operators, rich library functions, and extensibility**, C remains a popular and widely used programming language”

Algorithm

An algorithm is a procedure or formula for solving a problem.

Programs= algorithm + data

A program is a sequence of instructions to solve a problem. algorithm is the method used to solve the problem.

An algorithm is define as a complete , unambiguous , finite number (guaranteed to terminate)of logical steps for solving a specified problem.

It always provides an answer. It may not be the desired answer, but there will be an answer.

Each algorithm is a list of well-defined instructions for completing a task. Starting from an initial state(START), the instructions describe a computation that proceeds through a well-defined series of successive states, eventually terminating in a final ending state (STOP).

Guidelines for construct an algorithm

- ☐ It should begin with **start** statement
- ☐ Given problem should be broken down into simple and meaningful **steps**
- ☐ Steps should be **numbered** sequentially
- ☐ Steps should be **descriptive and written** in simple English
- ☐ Last step should be **end** statement

E.g.: Algorithm to average of n numbers

Step 1 : start

Step 2 : Read value of n

Step 3 : Read n numbers

Step 4 : add these n numbers

Step 5 : divide the sum by n to get average

Step 6 : print the average

Step 7 : Stop

Properties of Algorithm

- ☐ **Finiteness:** - an algorithm terminates after a **finite** numbers of steps.

- ❑ **Definiteness:** - each step in algorithm is unambiguous. This means that the action specified by the step cannot be interpreted (explain the meaning of) in multiple ways & can be performed without any confusion.
- ❑ **Input:-** an algorithm accepts zero or more inputs
- ❑ **Output:-** it produces at least one output
- ❑ **Effectiveness:-** it consists of basic instructions that are realizable. This means that the instructions can be performed by using the given inputs in a finite amount of time
- ❑ **Complete:-** The algorithm must work successfully in solving all the problems of a particular type for which it is designed.

E.g. Write an algorithm for finding the sum of two no's

Let the 2 no's be A & B and let their sum be equal to

C.

- 1.START
- 2.PRINT "ENTER 2 NUMBERS"
- 3.INPUT A,B
- 4.C=A+B
- 5.PRINT C
- 6.STOP

Flowchart

- A diagrammatic representation of sequential steps of solution of problem consisting standard block symbols being link together by directional arrows are called flowchart.
- *Program designing tool*
- Set of standard shaped boxes that are interconnected by flow lines
- Flow lines have arrows to indicate the flow of control between boxes
- The activity to be performed is written within the boxes in English
- Geometrical shapes used in flowchart are called flowchart symbols
- It facilitate communication between programmers and business person
- Easy to understand the logic
- Better documentation

Basic Flowchart symbols

Terminal



Input/output



Process



Flow lines



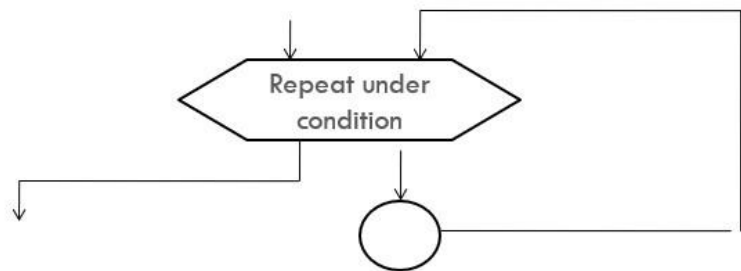
Decision



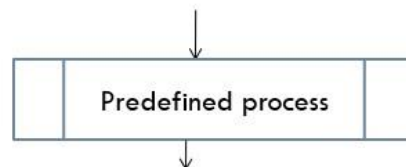
Connector



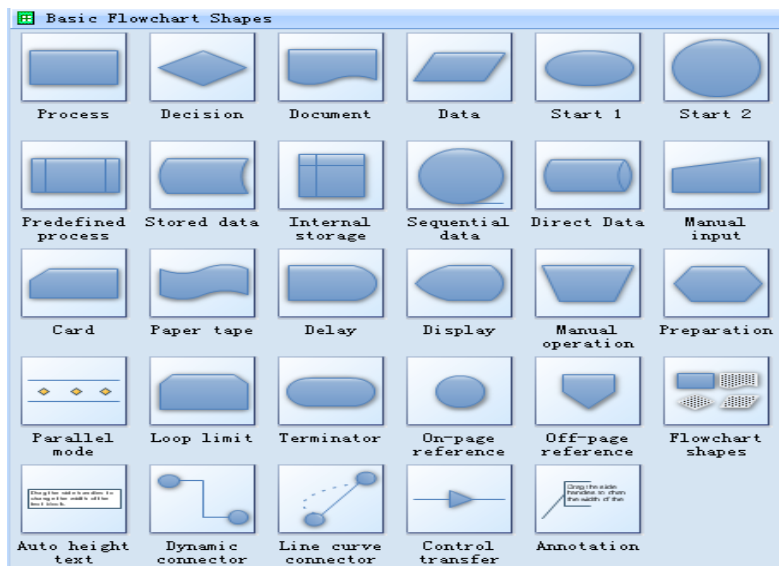
Repeat under
condition



Pre defined
process



Meaning of flowchart symbols



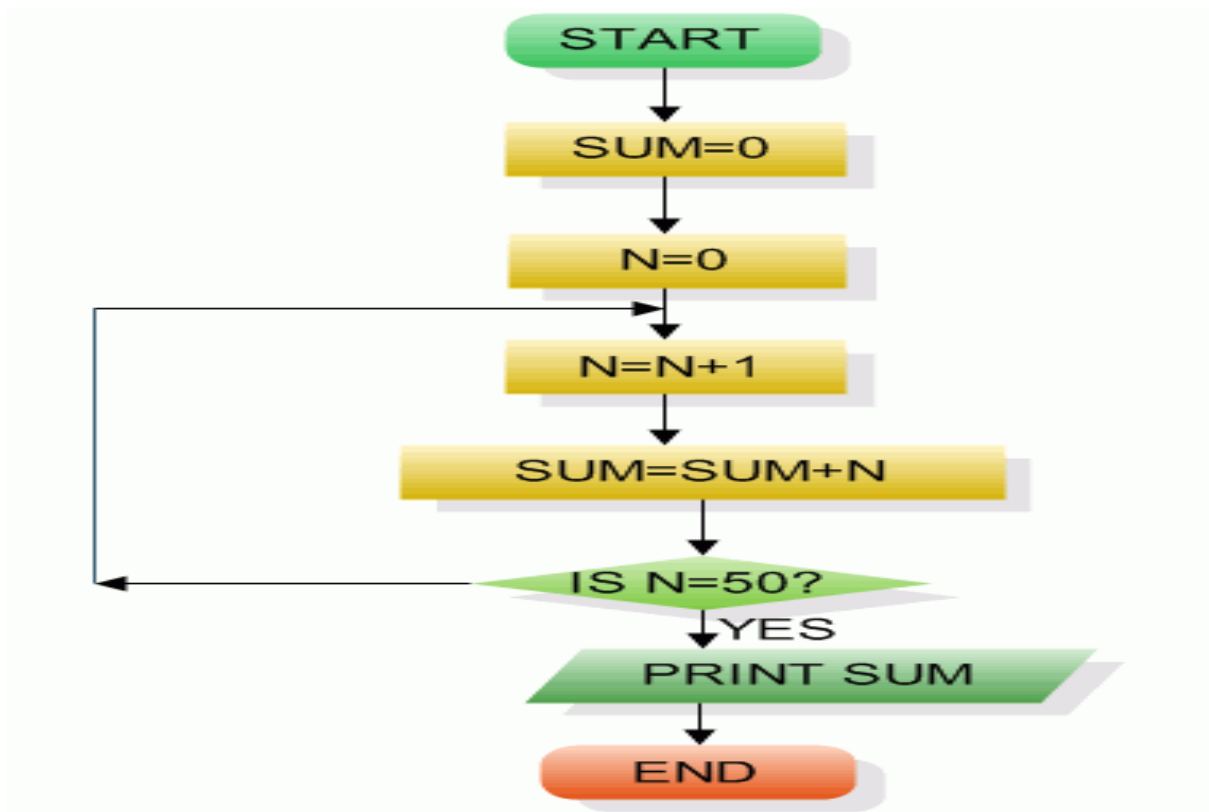
Symbol name	Description
Terminal	Indicates the beginning or end of an algorithm
Input/output	The point at which values of some data item have to read or some results written are indicated by input/output box
Process	Indicates computation or data manipulation(e.g. : assignment statements)
Flow lines	Connects the flowchart symbols and indicates the logic flow
Decision	Is used to indicate point at which decision has to be made and branch one/two alternatives possible (e.g.: if statement)
Connector	Indicates an entry to, or exit from another part of the flowchart or a connection point
Repeat under condition	Multiple branching (such as true or false . E.g.: switch, go to)

Guidelines for constructing a flowchart

- Flow charts start on the top of the page and flow down and to the left
- Use standard symbols only
- English should be used in flow charts, not programming languages and should be brief.
- Use arrow heads to indicate the direction.

- The flow chart for each subroutine must appear on a separate page
- While drawing flowchart Initially concentrate on the logic of the problem and draw the main path of the flowchart
- Flowchart can have only 1 logical start point and stop point
- Use descriptive term that aptly represent logic of the problem. Do not use ambiguous terms.
- It should be clear neat and easy to follow
- Remember that another user/programmer should easily understand flowchart
- Only flow line should emerge from a process symbol.
- Only one flow line should enter a decision symbol, but 2 or 3 flow lines can leave the decision symbol.
- Only flow line is used with terminal symbols
- Connector symbols should be used to reduce the no. of flow lines.
- The validity of flow chart should be tested by passing simple data through it.

Example of flowchart



Importance/ benefits of flowcharts

- ☐ Communication: Flowcharts are better way of communicating the logic of a system to all concerned.

- ☐ Effective analysis: With the help of flowchart, problem can be analyzed in more effective way.
- ☐ Proper documentation: Program flowcharts serve as a good program documentation, which is needed for various purposes.
- ☐ Efficient Coding: The flowcharts act as a guide or blueprint during the systems analysis and program development phase.
- ☐ Proper Debugging: The flowchart helps in debugging process.
- ☐ Efficient Program Maintenance: The maintenance of operating program becomes easy with the help of flowchart. It helps the programmer to put efforts more efficiently on that part.

Distinguish between algorithm and flow chart	
Algorithm	Flowchart
Algorithm is a procedure for solving a problem	Flowchart is a graphical diagram of the solution to the problem
The algorithm is constructed out of boxes and it is written as various steps	It is constructed with the shapes of each box indicating the kind of operation to be performed.
The actual operation are written in different steps	The actual operation to be performed is written inside the symbol
The step numbers shows which step is to be done next or 'go to' statements is used to specified the next step to be performed	The arrow coming out of symbol indicates which operation to perform next

Sample C Program – Printing a Message

A simple C program is used to print a message on the screen.

Example Program

```
#include <stdio.h>

int main()
{
    printf("I see, I remember");
    return 0;
}
```

Output

I see, I remember

Explanation of the Program

1. #include <stdio.h>

- `stdio.h` stands for **Standard Input Output Header**.
- It contains declarations for standard input/output functions such as `printf()` and `scanf()`.

2. main()

- `main()` is the **starting point of a C program**.
- Program execution begins from the `main()` function.
- Every C program must have **one main function**.
- The empty parentheses `()` indicate that the function has **no arguments**.

3. Opening and Closing Braces { }

- The opening brace `{` indicates the **beginning of the main function**.
- The closing brace `}` indicates the **end of the main function**.
- The statements between `{` and `}` form the **function body**.

4. Function Body

- The function body contains the instructions that perform the required task.
- In this example, the `printf()` statement is used to display the message.

5. Comments

- Comments are used to improve the **readability and understanding** of a program.
- Comments are ignored by the compiler and are **not executed**.
- A single-line comment can be written as:

```
// This is a comment
```

- A multi-line comment can be written as:

```
/*
```

This is a

multi-line comment

```
*/
```

- Comments can be placed at the beginning, middle, or end of a line.
- Comments **cannot be nested** in C.

6. printf()

- printf() is a standard C library function used to **display output on the screen**.
- The text to be printed is written inside **double quotation marks " "**.

Example:

```
printf("I see, I remember");
```

Output:

I see, I remember

7. Semicolon ;

- Every executable statement in C generally ends with a **semicolon ;**.

Example:

```
printf("Hello");
```

8. Printing Output in Two Lines

The `\n` character is used to move the cursor to the **next line**.

Example:

```
printf("I see,\n");
```

```
printf("I remember!");
```

Output:

I see,

I remember!

9. Newline Character \n

- `\n` is called the **newline character** or **escape sequence**.
- It moves the output to the next line.
- There should be **no space between \ and n**.

We can also use `\n` within a single printf() statement:

```
printf("I see,\nI remember!");
```

Output:

I see,

I remember!

10. Multiple Lines Using One printf()

A single printf() statement can print multiple lines by using \n.

Example:

```
printf("\nsee,\nI\nremember!");
```

Output:

I

see,

I

remember!

11. Case Sensitivity

- C is a **case-sensitive language**.
- Lowercase and uppercase letters are treated differently.

For example:

printf

and

PRINTF

are **not the same**.

- Keywords and standard function names are generally written in lowercase.
- Uppercase letters can be used in output strings.

Example:

```
printf("I SEE, I REMEMBER");
```

Output:

I SEE, I REMEMBER

The main() Function

The main() function is an important part of every C program because **program execution starts from main()**.

Some forms of main() are:

main()

int main()

void main()

main(void)

void main(void)

int main(void)

Meaning of void

- void inside the parentheses indicates that the function takes **no arguments**.
- void before main indicates that the function **does not return a value**.

Meaning of int

- int before main indicates that the function returns an **integer value** to the operating system.

Example:

```
int main()
```

```
{
```

```
    printf("Hello");
```

```
    return 0;
```

```
}
```

Here, return 0; indicates that the program has **completed successfully**.

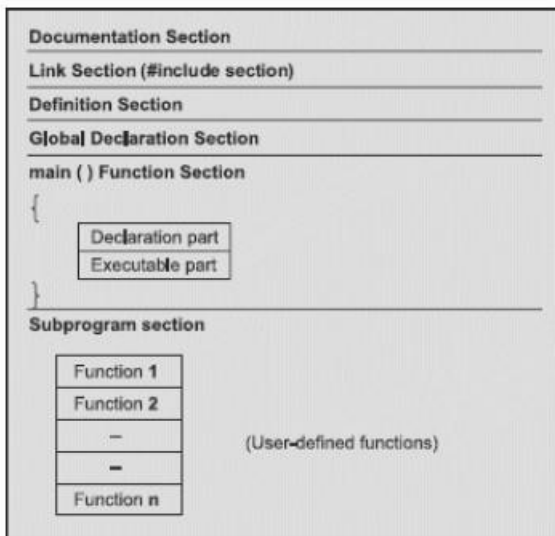
Basic Structure of C Programs

A C program can be viewed as a collection of **building blocks called functions**.

A **function** is a group of one or more statements designed to perform a **specific task**. To write a C program, we create functions and combine them to perform the required operation.

A C program may contain the following sections:

Basic Structure



. Documentation Section

- This section contains **comment lines**.
- It provides information about the program.
- It may include:
 - Program name
 - Author name
 - Date
 - Description of the program
- Comments are ignored by the compiler.

Example:

```
/* Program to find the sum of two numbers
```

```
Author: ABC
```

```
*/
```

. Link Section

- The link section provides instructions to the compiler to **include or link required library functions**.
- Header files are included using the #include directive.

Example:

```
#include <stdio.h>
```

Here, stdio.h provides standard input/output functions such as printf() and scanf().

. Definition Section

- This section is used to define **symbolic constants**.
- The #define preprocessor directive is commonly used.

Example:

```
#define PI 3.14
```

Here, PI is a symbolic constant with the value 3.14.

. Global Declaration Section

- Variables used by **more than one function** are called **global variables**.
- These variables are declared outside all functions.
- This section can also contain declarations of **user-defined functions**.

Example:

```
int count;
```

```
void display();
```

Here, count is a global variable and display() is a user-defined function declaration.

. main() Function Section

- Every C program must have **one main() function**.
- Program execution starts from the main() function.
- The main() function contains two parts:

a) Declaration Part

- This part declares the variables required by the program.

Example:

```
int a, b, sum;
```

b) Executable Part

- This part contains the **executable statements** that perform the required task.
- There must be at least one executable statement.

Example:

```
sum = a + b;
```

```
printf("%d", sum);
```

Both the declaration and executable parts are written between the **opening { and closing } braces**.

```
int main()
```

```
{
```

```

int a, b, sum;    // Declaration part
sum = a + b;     // Executable part
printf("%d", sum);
return 0;
}

```

- The opening brace { marks the beginning of the main() function body.
- The closing brace } marks the end of the main() function.
- The closing brace of main() represents the **logical end of the program**.

. Subprogram Section

- This section contains **user-defined functions**.
- These functions are called from the main() function or from other functions.
- User-defined functions are generally placed after the main() function.
- However, they can be placed in different orders depending on the program structure.

Example:

```

void display()
{
    printf("Hello");
}

```

General Format of a C Program

```

/* Documentation Section */

#include <stdio.h>    // Link Section

#define PI 3.14      // Definition Section

int x;               // Global Declaration Section

int main()           // main() Function Section
{
    int a, b;         // Declaration Part
    printf("Hello");  // Executable Part
    return 0;
}

/* Subprogram Section */

```

```
void display()
{
    printf("Welcome");
}
```

C Character Set

The **C character set** is a collection of characters that can be used to write **words, numbers, and expressions** in a C program.

The C character set is divided into **four main categories**:

. Letters

Letters include:

- **Uppercase letters:** A – Z
- **Lowercase letters:** a – z

Example:

A, B, C, ... Z

a, b, c, ... z

. Digits

Digits include:

0 1 2 3 4 5 6 7 8 9

They are used to represent **numeric values**.

Example:

```
int age = 20;
```

Here, 20 is formed using digits.

. Special Characters

Special characters are symbols used for different purposes in C programs.

Some commonly used special characters are:

+ - * / %

= < >

() { } []

; , . :

' " #

& | ! ?

—
For example:

- + → Addition
- - → Subtraction
- * → Multiplication
- / → Division
- = → Assignment
- ; → Statement terminator
- { } → Block of statements
- () → Function or expression grouping

. White Spaces

White spaces include:

- **Space**
- **Horizontal tab**
- **Vertical tab**
- **Newline**
- **Form feed**

White spaces are generally **ignored by the C compiler**, except when they are part of a **string constant**.

Example:

```
int a;
```

The space between int and a separates the **keyword** and **identifier**.

White spaces can be used to separate words, but they **cannot be placed inside keywords or identifiers**.

Correct:

```
int number;
```

Incorrect:

```
i nt number;
```

Trigraph Sequences(Not Important)

- Some keyboards, especially **non-English keyboards**, may not support all the special characters required in C.

- **ANSI C** introduced **trigraph sequences** to provide an alternative way to enter certain characters.
- A trigraph consists of **three characters**.
- It begins with **two question marks (??)** followed by another character.

Example

If a keyboard does not support square brackets [], they can be represented using trigraph sequences:

??(→ [

??) →]

Thus, trigraphs provide an alternative method for entering characters that may not be available on some keyboards.

C Tokens

In a passage of text, **individual words and punctuation marks** are called **tokens**.

In a C program, the **smallest individual units** are called **C tokens**.

A C program is written using **C tokens** according to the **syntax (rules) of the C language**.

Six Types of C Tokens

C has **six types of tokens**:

1. **Keywords**
2. **Identifiers**
3. **Constants**
4. **Strings**
5. **Operators**
6. **Special Symbols**

Example

Consider the statement:

```
int sum = a + 10;
```

Here:

- int → **Keyword**
- sum → **Identifier**
- = → **Operator**
- a → **Identifier**

- $+$ → **Operator**
- 10 → **Constant**
- ; → **Special Symbol**

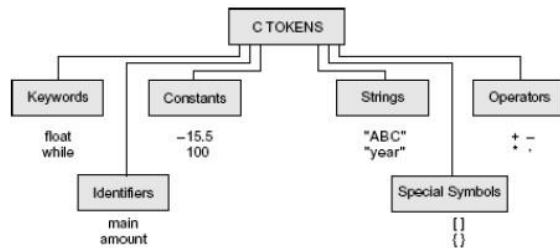


Fig. 3.1 C tokens and examples

Keywords and Identifiers in C

Every C word is classified as either a **keyword** or an **identifier**.

1. Keywords

- **Keywords** are reserved words in the C language.
- Each keyword has a **fixed meaning**.
- The meaning of a keyword **cannot be changed** by the programmer.
- Keywords are used as the **basic building blocks of C program statements**.
- All keywords must be written in **lowercase letters**.
- Some compilers may provide additional keywords.

Examples of C Keywords

int

float

char

if

else

for

while

do

switch

case

break

continue

return

void

struct

Example:

```
int age;
```

Here, int is a **keyword** because it specifies the data type of the variable age.

2. Identifiers

- **Identifiers** are names given by the programmer to different program elements.
- They are **user-defined names**.
- Identifiers can be used for:
 - Variables
 - Functions
 - Arrays
 - Structures, etc.

Example:

```
int studentAge;
```

Here, studentAge is an **identifier**.

Identifiers can contain:

- Letters (A-Z, a-z)
- Digits (0-9)
- Underscore (_)

Both **uppercase and lowercase letters** can be used. However, lowercase letters are commonly used.

Example:

student

student1

student_age

totalMarks

The underscore _ is commonly used to connect two or more words.

Example:

total_marks

student_name

Rules for Identifiers

The following rules must be followed when naming an identifier:

1. First Character

The first character must be an **alphabet (letter) or underscore (_)**.

Correct:

name

_student

total

Incorrect:

1student

2. Allowed Characters

An identifier can contain only:

- Letters
- Digits
- Underscore

Correct:

student1

total_marks

Incorrect:

student-name

3. First 31 Characters

According to traditional ANSI C rules, only the **first 31 characters** of an identifier are considered significant.

4. Cannot Be a Keyword

A keyword cannot be used as an identifier.

Incorrect:

int int;

Here, int is a keyword and cannot be used as a variable name.

5. No White Spaces

An identifier **must not contain spaces or other white spaces**.

Incorrect:

student name

Correct:

student_name

Keywords vs Identifiers

Keywords	Identifiers
Reserved words in C	User-defined names
Have fixed meanings	Meaning is defined by the programmer
Cannot be changed	Can be chosen by the programmer
Must be written in lowercase	Uppercase and lowercase can be used
Example: int, if, return	Example: age, total, student_name

Constants in C

A **constant** in C is a fixed value that **does not change during the execution of a program**.

C supports different types of constants:

1. **Integer Constants**
2. **Real (Floating-Point) Constants**
3. **Character Constants**
4. **String Constants**
5. **Backslash Character Constants (Escape Sequences)**

1. Integer Constants

An **integer constant** is a whole number without a fractional part.

C supports three types of integer constants:

a) Decimal Integer Constants

- Decimal integers use digits from **0 to 9**.
- They may have an optional + or - sign.

Examples:

15

-20

+100

5000

Important: Spaces, commas, and other non-digit characters are not allowed inside an integer.

Invalid examples:

15 750

20,000

\$1000

b) Octal Integer Constants

- Octal integers use digits from **0 to 7**.
- An octal integer starts with **0**.

Examples:

037

0435

0551

Note: Digits 8 and 9 are not allowed in octal numbers.

c) Hexadecimal Integer Constants

- Hexadecimal integers begin with **0x or 0X**.
- They use digits from **0 to 9** and letters from **A to F** or **a to f**.
- A to F represent values **10 to 15**.

Examples:

0X2

0x9F

0Xbcd

Integer Constant Prefixes

Type	Digits Used	Prefix	Example
Decimal	0–9	No prefix	25
Octal	0–7	0	037
Hexadecimal	0–9, A–F	0x or 0X	0x9F

Note: Octal and hexadecimal numbers are less commonly used in basic programming.

Integer Size and Qualifiers

The largest integer value that can be stored depends on the computer and data type.

Integer constants can be modified using qualifiers such as:

- U → Unsigned
- L → Long
- UL → Unsigned Long

Examples:

100U

100L

100UL

These qualifiers allow programmers to specify the required type of an integer constant.

2. Real Constants

Integer values cannot represent quantities that have **fractional parts**, such as:

- Distance
- Height
- Temperature
- Price

Such values are represented using **real constants** or **floating-point constants**.

Examples:

17.548

0.0083

-0.75

435.36

+247.0

A real constant can sometimes omit digits before or after the decimal point.

Examples:

215.

.95

-.71

+.5

All of these are valid real constants.

Exponential or Scientific Notation

A real number can also be written using **exponential notation**.

Example:

$$215.65 = 2.1565e2$$

Here:

- $2.1565 \rightarrow$ **Mantissa**
- $e \rightarrow$ Separates mantissa and exponent
- $2 \rightarrow$ **Exponent**

The general form is:

Mantissa e Exponent

The exponent may have a + or - sign.

Examples of valid floating-point constants:

0.65e4

12e-2

1.5e+5

3.18E3

-1.2E-1

Both lowercase e and uppercase E can be used.

Examples

$$7500000000 = 7.5E9$$

$$-0.000000368 = -3.68E-7$$

Exponential notation is useful for representing **very large or very small numbers**.

Important: White spaces are not allowed inside a floating-point constant.

Floating-Point Suffixes

Floating-point constants are normally treated as **double-precision** values.

Suffixes can be used to specify the type:

- f or F $\rightarrow$ Single precision
- l or L $\rightarrow$ Extended precision

Examples:

3.14f

3.14F

3.14L

3. Single Character Constants

A **character constant** contains a single character enclosed in **single quotation marks** (' ').

Examples:

'5'

'X'

','

' '

The last example represents a **blank space**.

Character Constant vs Integer Constant

'5' → Character constant

5 → Integer constant

They are **not the same**.

- '5' represents the character **5**.
- 5 represents the numerical value **5**.

ASCII Values

Each character constant has an integer value called its **ASCII value**.

For example:

'a' → ASCII value 97

The ASCII value of a can be displayed using a suitable printf() statement.

Since character constants have integer values, **arithmetic operations can also be performed on them**.

4. String Constants

A **string constant** is a sequence of characters enclosed within **double quotation marks** (" ").

The characters in a string can include:

- Letters
- Numbers
- Special characters
- Spaces

Examples:

"Hello!"

"1987"

"WELL DONE"

"?...!"

"5+3"

"X"

Character Constant vs String Constant

'X' → Character constant

"X" → String constant

They are different.

5. Backslash Character Constants / Escape Sequences

C supports special character combinations beginning with a **backslash (\)**.

These are called **escape sequences** or **backslash character constants**.

They are commonly used with output functions such as printf().

Common Escape Sequences

Escape Sequence Meaning

\n	Newline
\t	Horizontal tab
\b	Backspace
\r	Carriage return
\f	Form feed
\a	Alert/Bell
\\	Backslash
\'	Single quote
\"	Double quote
\0	Null character

Example:

```
printf("Hello\nWorld");
```

Output:

Hello

World

Here, \n moves the output to the **next line**

Variables in C

A **variable** is a name given to a memory location that is used to **store a data value**.

Unlike a **constant**, whose value does not change during program execution, a **variable can store different values at different times** during the execution of a program.

Example

```
int amount;
```

```
amount = 5000;
```

```
amount = 6000;
```

Here, amount is a variable. Its value changes from 5000 to 6000.

Variable Names

A variable name is chosen by the programmer. It is better to choose a **meaningful name** that represents the purpose of the variable.

Examples

Average

height

Total

Counter_1

class_strength

For example:

```
int class_strength;
```

Here, class_strength clearly indicates that the variable stores the strength of a class.

Rules for Naming Variables

Variable names can contain:

- Letters (A-Z, a-z)
- Digits (0-9)
- Underscore (_)

The following rules must be followed:

1. Must Begin with a Letter

A variable name must normally begin with an **alphabet (letter)**.

Example:

total

average

student1

Some systems also allow an underscore as the first character.

Example:

_total

2. Length of Variable Name

- Traditional ANSI C recognizes up to **31 significant characters** in an identifier.
- In **C99**, at least **63 characters** are significant.

3. Case Sensitive

C is a **case-sensitive language**.

The following are three different variable names:

Total

total

TOTAL

They are treated as **different variables**.

4. Cannot Be a Keyword

A keyword cannot be used as a variable name.

Incorrect:

```
int int;
```

Here, int is a C keyword, so it cannot be used as a variable name.

5. No White Space

White spaces are not allowed in variable names.

Incorrect:

```
class strength
```

Correct:

```
class_strength
```

Examples of Valid Variable Names

age
total
average
student1
marks_1
class_strength
_total

Examples of Invalid Variable Names

1student // Starts with a digit
class strength // Contains white space
total-marks // Contains special character -
int // Keyword
student@ // Contains special character @

Variable Declaration

Before using a variable, it should be **declared** with its data type.

Example

```
int age;  
float average;  
char grade;
```

Here:

- age → Integer variable
- average → Floating-point variable
- grade → Character variable

Declaration and Assignment

```
int age;  
age = 20;
```

Or, both can be done together:

```
int age = 20;
```

Difference Between Constants and Variables

Constants

Value does not change during program execution

Example: 10

Represents a fixed value

Example: $\pi = 3.14$

Variables

Value can change during program execution

Example: age

Stores a changeable value

Example: marks = 50, later marks = 75

Data Types in C

C is a **richly typed programming language**. It provides different data types so that the programmer can select a suitable type based on the requirements of the application.

The size and storage representation of data types may vary depending on the **computer and compiler**.

Classification of Data Types in C

ANSI C supports three main classes of data types:

1. **Primary (Fundamental) Data Types**
2. **Derived Data Types**
3. **User-Defined Data Types**

1. Primary Data Types

The basic or fundamental data types are:

- int – Integer
- char – Character
- float – Floating-point number
- double – Double-precision floating-point number
- void – No value

2. Derived Data Types

Derived data types include:

- Arrays
- Functions
- Structures
- Pointers

3. User-Defined Data Types

These are data types created by the programmer according to the requirements of the program.

1. Integer Data Types

Integers are **whole numbers** without a fractional part.

Examples:

10

-25

1000

The range of integer values depends on the **machine and compiler**.

For example, on a **16-bit machine**, a signed integer generally has the range:

-32768 to +32767

This is calculated as:

-2^{15} to $2^{15} - 1$

On a **32-bit machine**, the range is generally:

-2,147,483,648 to 2,147,483,647

Types of Integer Data

C provides three main integer types:

1. short int
2. int
3. long int

These can be used in both **signed** and **unsigned** forms.

Signed Integer

- A signed integer can store **positive and negative values**.
- The signed qualifier is optional because integers are signed by default.

Example:

```
signed int age;
```

or

```
int age;
```

Unsigned Integer

- An unsigned integer stores **only non-negative values**.
- It uses all available bits to represent the magnitude of the number.
- Therefore, it can store a larger positive range than a signed integer of the same size.

For example, on a 16-bit machine:

Unsigned integer range: 0 to 65,535

Example:

```
unsigned int count;
```

Integer Qualifiers

The following qualifiers are commonly used with integer types:

- short
- long
- signed
- unsigned

Examples:

```
short int a;
```

```
int b;
```

```
long int c;
```

```
unsigned int d;
```

These qualifiers provide control over the **range of values and storage requirements**.

2. Floating-Point Data Types

Floating-point data types are used to represent **real numbers** or numbers with fractional parts.

Examples:

3.14

-5.25

25.75

C provides three main floating-point types:

1. float
2. double
3. long double

a) float

- Used to store floating-point numbers.
- Generally occupies **32 bits**.
- Provides approximately **6 digits of precision** in the traditional model.

Example:

```
float temperature;
```

b) double

- Used when greater accuracy is required than float.
- Generally occupies **64 bits**.
- Provides approximately **14–15 digits of precision** in the traditional model.
- It is called a **double-precision** floating-point type.

Example:

```
double salary;
```

c) long double

- Used when still greater precision is required.
- Traditionally associated with **80-bit** representation on some systems.
- The actual size and precision are **compiler and platform dependent**.

Example:

```
long double value;
```

Comparison

Data Type	Purpose	Traditional Size
float	Single precision	32 bits
double	Double precision	64 bits
long double	Extended precision	Often 80 bits

Note: The exact size and range of floating-point types can vary depending on the compiler and computer system.

3. Void Data Type

The void type represents **no value**.

It is commonly used with functions.

Example

```
void display()  
{  
    printf("Hello");  
}
```

Here, void indicates that the function **does not return a value**.

The void type can also be used as a generic type in certain C programming contexts, such as void *.

4. Character Data Type

A single character is stored using the **char** data type.

Examples:

'A'

'5'

'X'

Example:

```
char grade;
```

A character typically occupies **1 byte (8 bits)** of storage.

The char type can be:

- signed char
- unsigned char

Signed Char

Traditionally, a signed 8-bit character has a range of:

-128 to 127

Unsigned Char

An unsigned 8-bit character has a range of:

0 to 255

Examples:

```
signed char a;
```

```
unsigned char b;
```

Data Type Description		Example
int	Stores whole numbers	int age = 20;
char	Stores a single character	char grade = 'A';
float	Stores single-precision real numbers	float price = 25.5;
double	Stores double-precision real numbers	double value = 3.14159;

Data Type Description

Example

void Represents no value

void display();

Declaration of Variables in C

After selecting suitable variable names, the variables must be **declared to the compiler** before they are used.

Purpose of Variable Declaration

A variable declaration tells the compiler two important things:

1. **The name of the variable**
2. **The type of data the variable can store**

Example:

```
int age;
```

Here:

- int → Data type
- age → Variable name

. Primary Type Declaration

The general syntax for declaring variables is:

```
data_type variable1, variable2, ..., variableN;
```

- data_type specifies the type of data.
- variable1, variable2, ... are the names of variables.
- Multiple variables are separated by **commas**.
- A declaration statement ends with a **semicolon ;**.

Examples

```
int age;
```

```
float average;
```

```
double salary;
```

```
char grade;
```

Multiple variables can be declared in one statement:

```
int a, b, c;
```

Here, a, b, and c are all integer variables.

. Where to Declare Variables?

Variables are generally declared **before they are used**.

Example:

```
int main()
{
    int age;
    float average;

    age = 20;
    average = 75.5;

    return 0;
}
```

In traditional C programming, declarations were commonly placed at the beginning of a block.

Variables can also be declared outside the main() function. Such variables are generally called **global variables**.

Example:

```
int count;    // Global variable

int main()
{
    int age;    // Local variable

    return 0;
}
```

. Data Type Keywords

Some common data types and their keywords are:

Data Type	Keyword
Integer	int
Character	char

Data Type	Keyword
------------------	----------------

Floating point	float
----------------	-------

Double precision	double
------------------	--------

No value	void
----------	------

. Use of Qualifiers

The qualifiers short, long, and unsigned can be used with basic data types.

If short, long, or unsigned is used without specifying a basic integer type, it is generally treated as an int type.

Examples:

```
short a;
```

```
long b;
```

```
unsigned c;
```

These are generally equivalent to:

```
short int a;
```

```
long int b;
```

```
unsigned int c;
```

If a character variable is to be declared as unsigned, both keywords should be specified:

```
unsigned char ch;
```

. Default Types of Constants

Integer Constants

Integer constants are normally treated as **int** type.

Example:

```
100
```

It is normally an int constant.

The type can be modified using suffixes:

- U or u → Unsigned
- L or l → Long

Examples:

```
100U
```

```
100L
```

100UL

Floating-Point Constants

Floating-point constants are normally treated as **double** type.

Example:

3.14

To specify a float constant, use f or F:

3.14f

3.14F

To specify a long double constant, use l or L:

3.14L

Summary

Constant Default Type Suffix

10	int	U, L, UL
3.14	double	f/F → float
3.14L	long double	L

. User-Defined Type Declaration – typedef

C provides the typedef keyword to create a **new name (alias) for an existing data type**.

Syntax

```
typedef existing_type new_name;
```

Examples

```
typedef int units;
```

```
typedef float marks;
```

Here:

- units represents int
- marks represents float

They can then be used to declare variables:

```
units batch1, batch2;
```

```
marks mark1, mark2;
```

Here:

- batch1 and batch2 are int variables.

- mark1 and mark2 are float variables.

Advantage of typedef

The main advantage of typedef is that it allows programmers to create **meaningful data type names**, which improves the **readability and understandability** of programs.

Important Point

typedef does **not create a new data type**. It only creates a new name (alias) for an existing data type.

. Enumerated Data Type – enum

An **enumerated data type** is a user-defined data type that consists of a set of named integer constants.

It is declared using the keyword enum.

General Syntax

```
enum identifier  
{  
    value1, value2, value3, ... valueN  
};
```

The names inside { } are called **enumeration constants**.

Variables of the enumerated type can store one of these defined values.

Example

```
enum day  
{  
    Monday,  
    Tuesday,  
    Wednesday,  
    Thursday,  
    Friday,  
    Saturday,  
    Sunday  
};
```

Basic Input and Output in C

Input and Output (I/O) operations are fundamental in [C programming](#) as they allow programs to interact with users and external devices. Input operations are used to receive data from the user, while output operations display information on the screen or write data to files.

- Enables communication between the program and the user.
- Provided through the Standard Input/Output library (<stdio.h>).
- Supports formatted and unformatted input/output operations.

Output

Output operations in C are used to display data on the screen or write data to files. The most commonly used output function is `printf()`, which allows formatted output. Other functions such as `fputs()` can also be used to display strings.

- Supports formatted output using format specifiers.
- Can print text, variables, expressions, and strings.

The following examples demonstrate the use of `printf` for output in different cases:

Printing Some Text

```
#include <stdio.h>

int main() {
    // Prints some text
    printf("First Print");
    return 0;
}
```

OUTPUT

First print

Printing Variables

```
#include <stdio.h>

int main() {
    int age = 22;
    // Prints Age
    printf("%d\n", age);
    return 0;
}
```

Output

Printing Variables Along with String

```
#include <stdio.h>

int main() {
    int age = 22;

    // Prints Age
    printf("The value of the variable age is %d\n", age);

    return 0;
}
```

Output

The value of the variable age is 22

Input

Input operations in C are used to receive data from users during program execution. Functions such as `scanf()`, `fgets()`, and `getchar()` allow programs to read different types of input, including numbers, characters, and strings.

- Supports multiple data types such as integers, characters, and strings.
- Provides both formatted (`scanf()`) and safer string input methods (`fgets()`).

Syntax

```
scanf("formatted_string", address_of_variables);
```

Remember that this function takes the address of the arguments where the read value is to be stored.

Examples of Reading User Input

Reading an Integer

```
#include <stdio.h>

int main() {
    int age;

    printf("Enter your age: ");

    // Reads an integer
    scanf("%d", &age);

    // Prints the age
    printf("Age is: %d\n", age);
}
```

```
    return 0;
}
```

Output

Enter your age:
25 *(Entered by the user)*
Age is: 25

Reading a Character

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: \n");
    // Reads an Character
    scanf("%c", &ch);
    // Prints the Character
    printf("Entered character is: %c\n", ch);
    return 0;
}
```

Output

Enter a character:
a *(Entered by the user)*
Entered character is: a

Reading a string

The scanf() function can also be used to read string input from users. But it can only read single words.

```
#include <stdio.h>

int main() {
    char str[100]; // Declare an array to hold the input string
    printf("Enter a string: ");
    scanf("%s", str); // Reads input until the first space or newline
    printf("You entered: %s\n", str);
}
```

```
    return 0;
}
```

Output:

Enter a String:

Geeks (Entered by the user)

Entered string is: Geeks

The scanf() function with %s stops at the first whitespace and does not read spaces. To handle this, we can use [fgets\(\)](#), which can read spaces and is safer because it limits the number of characters read, helping prevent buffer overflow.

Operators in C

Operators in C are special symbols used to perform operations on variables, constants, and expressions. They form the foundation of programming logic by enabling arithmetic calculations, comparisons, logical decisions, memory access, and bit-level manipulations.

- C supports arithmetic, relational, logical, bitwise, assignment, and special operators.
- Operators can be unary, binary, or ternary based on the number of operands.

Types of C Operators

C operators are classified into the following categories:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Increment and Decrement Operators
6. Conditional Operator
7. Bitwise Operators
8. Special Operators

Arithmetic Operators in C

Arithmetic operators in C are used to perform mathematical calculations on numeric values such as integers and floating-point numbers. They enable operations like addition, subtraction, multiplication, division, and modulus, making them essential for performing calculations, processing data, and solving computational problems in C programs.

- Support both integer and floating-point calculations.
- Classified into binary and [unary arithmetic operators](#).

```
#include <stdio.h>

int main() {
    // Calculate the area of the triangle

    int sum = 10 + 20;

    printf("%d", sum);

    return 0;
}
```

Output

30

Explanation: In this example, the addition operator (+) adds the values 10 and 20. The result 30 is stored in the variable sum and displayed using printf().

Arithmetic operators can be classified into two types based on the number of operands

Binary Arithmetic Operators

Binary arithmetic operators work on two operands and perform mathematical operations such as addition, subtraction, multiplication, division, and modulus.

- Perform basic mathematical calculations.
- Return a numeric result.
- Work with both variables and constants.

The binary arithmetic operators work on two operands. C provides 5 such operators for performing arithmetic functions which are as follows:

Name	Operator	Arithmetic Operation	Syntax
Addition	+	Add two operands.	$x + y$
Subtraction	-	Subtract the second operand from the first operand.	$x - y$
Multiplication	*	Multiply two operands.	$x * y$
Division	/	Divide the first operand by the second operand.	x / y
<u>Modulus</u>	%	Calculate the remainder when the first operand is divided by the second operand.	$x \% y$

```
#include <stdio.h>

int main()
{
```

```

int a = 10, b = 4, res;

// Addition

res = a + b;

printf("a + b is %d\n", res);

// Subtraction

res = a - b;

printf("a - b is %d\n", res);

// Multiplication

res = a * b;

printf("a * b is %d\n", res);

// Division

res = a / b;

printf("a / b is %d\n", res);

// Modulus

res = a % b;

printf("a %% b is %d\n", res);

return 0;
}

```

Output

a + b is 14

a - b is 6

a * b is 40

a / b is 2

a % b is 2

Unary Arithmetic Operators

Unary arithmetic operators work on a single operand and are mainly used to increase, decrease, or change the sign of a value.

- Include increment (++) and decrement (--) operators.
- Support unary plus (+) and unary minus (-).
- Useful for counters and loop control.

The unary arithmetic operators work with a single operand. In C, we have four such operators which are as follows:

Name	Operator	Description	Syntax
Decrement	--	Decreases the integer value of the variable by one.	--a or a--
Increment	++	Increases the integer value of the variable by one.	++a or a++
Unary Plus	+	Returns the value of its operand.	+a
Unary Minus	-	Returns the negative of the value of its operand.	-a

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10, res;
```

```
    // post-incrementing a res is assigned 10, a is not updated yet
```

```
    res = a++;
```

```
    printf("a is %d, res is %d\n", a, res);
```

```
    // post-decrementing res is assigned 11, a is not updated yet
```

```
    res = a--;
```

```
    printf("a is %d, res is %d\n", a, res);
```

```
    res = ++a;
```

```
    // a and res have same values = 11
```

```
    printf("a is %d, res is %d\n", a, res);
```

```
    res = --a;
```

```
    // a and res have same values = 10
```

```
    printf("a is %d, res is %d\n", a, res);
```

```
    printf("+a is %d\n", +a);
```

```
    printf("-a is %d", -a);
```

```
    return 0;
```

```
}
```

output

a is 11, res is 10

a is 10, res is 11

a is 11, res is 11

a is 10, res is 10

+a is 10

-a is -10

Explanation: In the given example, postfix operators (a++, a--) first assign the current value and then update the variable. Prefix operators (++a, --a) update the variable first and then use the updated value. Unary plus returns the value unchanged, while unary minus returns its negative value.

Increment and Decrement Operators in C

The **increment (++)** and **decrement (--)** operators in C are unary operators for incrementing and decrementing the numeric values **by 1**, respectively. They are one of the most frequently used operators in programming for looping, array traversal, pointer arithmetic, and many more.

Increment Operator in C

The **increment operator (++)** is used to increment the value of a variable in an expression by 1. It can be used on variables of the numeric type, such as integer, float, character, pointers, etc.

The Increment Operator in C can be used in two ways, either as a prefix (pre-increment) or a postfix (post-increment).

Syntax:

// AS PREFIX

++m

// AS POSTFIX

m++

where m is variable. Both pre-increment and post-increment increase the value of the variable but there is a little difference in how they work.

1. Pre-Increment

In pre-increment, the increment operator is used as the prefix. Also known as prefix increment, the value is incremented first according to the precedence and then the less priority operations are done.

result = ++var1;

The above expression can be expanded as

```
var = var + 1;
```

```
result = var;
```

2. Post-Increment

In post-increment, the increment operator is used as the suffix of the operand. The increment operation is performed after all the other operations are done. It is also known as postfix increment.

```
result = var1++;
```

The above expression is equivalent

```
result = var;
```

```
var = var + 1;
```

Decrement Operator in C

The decrement operator is used to decrement the value of a variable in an expression. In the Pre-Decrement, the value is first decremented and then used inside the expression. Whereas in the Post-Decrement, the value is first used inside the expression and then decremented.

Syntax

Just like the increment operator, the decrement operator can also be used in two ways:

// AS PREFIX

```
--m
```

// AS POSTFIX

```
m--
```

where m is variable.

1. Pre-Decrement Operator

The pre-decrement operator decreases the value of the variable immediately when encountered. It is also known as prefix decrement as the decrement operator is used as the prefix of the operand.

```
result = --m;
```

which can be expanded to

```
m = m - 1;
```

```
result = m;
```

2. Post-Decrement Operator

The post-decrement happens when the decrement operator is used as the suffix of the variable. In this case, the decrement operation is performed after all the other operators are evaluated.

```
result = m--;
```

The above expression can be expanded as

```
result = m;
```

```
m = m-1;
```

Increment Operator	Decrement Operator
Increment Operator adds 1 to the operand.	Decrement Operator subtracts 1 from the operand.
The Postfix increment operator means the expression is evaluated first using the original value of the variable and then the variable is incremented(increased).	The Postfix decrement operator means the expression is evaluated first using the original value of the variable and then the variable is decremented(decreased).
Prefix increment operator means the variable is incremented first and then the expression is evaluated using the new value of the variable.	Prefix decrement operator means the variable is decremented first and then the expression is evaluated using the new value of the variable.
Generally, we use this in decision-making and looping.	This is also used in decision-making and looping.

Increment Operator (++)

The [increment operator \(++ \)](#) is used to increment the value of the variable by 1. The increment can be done in two ways:

A. Prefix Increment

In this method, the operator precedes the operand (e.g., ++a). The value of the operand will be altered *before* it is used. For example:

```
int a = 1;
```

```
int b = ++a; // b = 2
```

B. Postfix Increment

In this method, the operator follows the operand (e.g., a++). The value operand will be altered *after* it is used. For example:

```
int a = 1;
int b = a++; // b = 1
int c = a; // c = 2
```

Below example shows the implementation of **increment (++)**:

```
#include <stdio.h>

int main(){
    int a = 1;
    int b = 1;

    printf("Pre-Incrementing a = %d\n", ++a);
    printf("Post-Incrementing b = %d", b++);

    return 0;
}
```

Output

Pre-Incrementing a = 2

Post-Incrementing b = 1

Decrement Operator (--)

The [decrement operator \(-- \)](#) is used to decrement the value of the variable by 1. The decrement can be done in two ways:

A. Prefix Decrement

In this method, the operator precedes the operand (e.g., $-a$). The value of the operand will be altered *before* it is used. For example:

```
int a = 1;
int b = --a; // b = 0
```

B. Postfix Decrement

In this method, the operator follows the operand (e.g., $a-$). The value of the operand will be altered *after* it is used. For example:

```
int a = 1;
int b = a--; // b = 1
int c = a; // c = 0
```

Below example shows the implementation of **decrement (--)**:

```
#include <stdio.h>

int main() {
    int a = 1;
```

```

int b = 1;

printf("Pre-Decrementing a = %d\n", --a);

printf("Post-Decrementing b = %d", b--);

return 0;

}

```

Output

Pre-Decrementing a = 0

Post-Decrementing b = 1

Unary Plus

The unary plus (+) operator does not change the sign of its argument; it simply returns the value as is. It is often used for code clarity rather than functionality.

```

int a = -10;
int b = +a; // b = -10

```

The unary plus is different from the addition operator, as addition requires two operands.

Below is the implementation of the unary plus (+) operator:

```

#include <stdio.h>

int main() {
    // Declaring a negative integer
    int a = -10;

    // Using unary plus to keep value unchanged
    int b = +a;

    printf("%d\n", a);
    printf("%d", b);

    return 0;
}

```

Output

-10

-10

Unary Minus

The **minus operator** (-) changes the sign of its argument. A positive number becomes negative, and a negative number becomes positive.

```
int a = 10;
int b = -a; // b = -10
```

Unary minus is different from the subtraction operator, as subtraction requires two operands.

Below is the implementation of the **unary minus (-)** operator:

```
#include <stdio.h>

int main() {
    // declaring a positive integer
    int a = 10;
    // using - sign to make the value of positive integers
    // to negative
    int b = -a;
    printf("%d\n", a);
    printf("%d", b);
    return 0;
}
```

Output

10

-10

Relational Operators in C

Relational operators in C are used to compare two values or expressions and determine the relationship between them. They are commonly used in decision-making statements such as if, if-else, while, and for loops to control the flow of a program based on conditions.

- Used to compare two operands or expressions.
- Always return 1 (true) or 0 (false).
- Commonly used in conditional and looping statements.

Types of C Relational Operators

There are a total of 6 relational operators in C language. There are:

Relational Operators in C

Operator Symbol	Operator Name
==	Equal To
!=	Not Equal To
<	Less Than
>	Greater Than
<=	Less Than Equal To
>=	Greater Than Equal To

1. Equal to operator (==)

The C equal to operator (==) is a relational operator that is used to check whether the two given operands are equal or not.

- Equal to operator is a binary operator hence it requires two operands to perform the comparison.
- If the two values are equal, it returns true. Otherwise, it returns false.

Syntax

operand1 == operand2

For example, 5==5 will return true.

2. Not equal to operator (!=)

The C not equal (==) to operator is another relational operator used for checking whether the two given operands are equal or not.

- It is also a binary operator, requiring two operands to perform the comparison.
- It is the exact boolean complement of the '==' operator which returns true if the two values are not equal, false otherwise.

Syntax

operand1 != operand2

For example, 5!=5 will return false.

3. Greater than operator (>)

The greater than operator is a relational operator in C that checks whether the first operand is greater than the second operand or not.

- If the operand first is greater than the operand2, it returns true. Otherwise, it returns false.

- This operator is used to make decisions or create conditions based on the relative magnitude of two values.

Syntax

operand1 > operand2

For example, 6 > 5 will return true.

4. Less than operator (<)

The less than operator is a relational operator in C that checks whether the first operand is lesser than the second operand.

- If the operand first is less than the operand2, it returns true. Otherwise, it returns false.
- This operator is also used to make decisions or create conditions based on the relative magnitude of two values.

Syntax

operand1 < operand2

For example, 6 < 5 will return false.

Note: *The greater than and less than operator are not equal to the complement of each other.*

5. Greater than or equal to operator (>=)

The greater than or equal to the operator is a relational operator in C that checks whether the first operand is greater than or equal to the second operand.

- It is a binary operator.
- If the first operand is greater than or equal to the second operand, it returns true. Otherwise, it returns false.

Syntax

operand1 >= operand2

For example, 5 >= 5 will return true.

6. Less than or equal to the the operator (<=)

The less than or equal to the operator is a relational operator in C that checks whether the first operand is less than or equal to the second operand.

- It is a binary operator.
- If the first operand is less than or equal to the second operand, it returns true. Otherwise, it returns false.

Syntax

operand1 <= operand2

For example, 5 <= 5 will also return true.

Example of Relational Operator in C

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10, b = 4;
```

```
    // greater than example
```

```
    if (a > b)
```

```
        printf("a is greater than b\n");
```

```
    else
```

```
        printf("a is less than or equal to b\n");
```

```
    // greater than equal to
```

```
    if (a >= b)
```

```
        printf("a is greater than or equal to b\n");
```

```
    else
```

```
        printf("a is lesser than b\n");
```

```
    // less than example
```

```
    if (a < b)
```

```
        printf("a is less than b\n");
```

```
    else
```

```
        printf("a is greater than or equal to b\n");
```

```
    // lesser than equal to
```

```
    if (a <= b)
```

```
        printf("a is lesser than or equal to b\n");
```

```
    else
```

```
        printf("a is greater than b\n");
```

```
    // equal to
```

```
    if (a == b)
```

```
        printf("a is equal to b\n");
```

```
    else
```

```

        printf("a and b are not equal\n");
// not equal to
if (a != b)
    printf("a is not equal to b\n");
else
    printf("a is equal b\n");
return 0;
}

```

Output

```

a is greater than b
a is greater than or equal to b
a is greater than or equal to b
a is greater than b
a and b are not equal
a is not equal to b

```

Explanation: In this program, different relational operators are used to compare the values of a (10) and b (4). Since 10 is greater than 4, the conditions $a > b$, $a \geq b$, and $a \neq b$ evaluate to true, while $a < b$, $a \leq b$, and $a == b$ evaluate to false. Based on these results, the corresponding messages are printed to the console.

C Logical Operators

Logical operators in C are used to combine multiple conditions and evaluate them as a single expression. They are commonly used in decision-making statements such as if, if-else, while, and for loops to control program flow.

- Logical operators return either 1 (true) or 0 (false) based on the evaluation of the expression.
- They help in combining, negating, or testing multiple conditions efficiently.

Types of Logical Operators in C

C provides three logical operators:

1. Logical AND Operator (&&)

The logical AND operator (&&) returns true only if both operands are non-zero. Otherwise, it returns false (0). The return type of the result is int. Below is the truth table for the logical AND operator.

X	Y	X && Y
1	1	1
1	0	0
0	1	0
0	0	0

Syntax

(operand_1 && operand_2)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10, b = 20;
```

```
    if (a > 0 && b > 0)
```

```
    {
```

```
        printf("Both values are greater than 0\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("One or both values are less than or equal to 0\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Output

Both values are greater than 0

2. Logical OR Operator (||)

The logical OR operator returns true if any one of the operands is non-zero. Otherwise, it returns false i.e., 0 as the value. Below is the truth table for the logical OR operator.

X	Y	X Y
1	1	1
1	0	1
0	1	1
0	0	0

Syntax

(operand_1 || operand_2)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = -1, b = 20;
```

```
    if (a > 0 || b > 0)
```

```
    {
```

```
        printf("Any one of the given value is "
```

```
            "greater than 0\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("Both values are less than or equal to 0\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Output

Any one of the given value is greater than 0

3. Logical NOT Operator (!)

The logical NOT operator (!) reverses the result of a condition. If the condition is true, it becomes false, and if the condition is false, it becomes true. Below is the truth table for the logical NOT operator.

X	!X
0	1
1	0

Syntax

```
!(operand_1 && operand_2)
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10, b = 20;
```

```
    if (!(a > 0 && b > 0))
```

```
    {
```

```
        // condition returned true but logical NOT operator changed it to false
```

```
        printf("Both values are greater than 0\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("Both values are less than 0\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Assignment Operators in C

In C, assignment operators are used to assign values to variables. The left operand is the variable and the right operand is the value being assigned. The value on the right must match the data type of the variable otherwise, the compiler will raise an error.

```
#include <stdio.h>

int main()
{

    // Assigning value 10 to a using "=" operator

    int a = 10;

    printf("%d", a);

    return 0;

}
```

Output

10

Explanation: In the above example, the assignment operator (=) is used to assign the value **10** to the variable **a**. The **printf()** function then prints the value of a, which is 10, to the console.

Compound Assignment Operators

C also provides compound assignment operators that combine an operation and assignment in a single step. They make the code shorter and more efficient.

Here are the most commonly used compound assignment operators:

1. Addition Assignment (+=)

Adds the value of the right operand to the left operand and stores the result in the left operand.

```
#include <stdio.h>

int main()
{

    int a = 5;

    // Equivalent to a = a + 3
```

```
a += 3;
printf("%d", a);
return 0;
}
```

Output

a = 8

2. Subtraction Assignment (-=)

Subtracts the value of the right operand from the left operand and stores the result in the left operand.

```
#include <stdio.h>
int main()
{
    int a = 10, b = 5;

    // a = a - b
    a -= b;
    printf("%d", a);
    return 0;
}
```

Output

5

3. Multiplication Assignment (*=)

Multiplies the value of the right operand by the left operand and stores the result in the left operand.

```
#include <stdio.h>
int main()
{
    int a = 10, b = 5;

    // a = a * b
```

```
a *= b;
printf("%d", a);
return 0;
}
```

Output

50

4. Division Assignment (/=)

Divides the left operand by the right operand and stores the result in the left operand.

```
#include <stdio.h>
```

```
int main()
{
    int a = 10, b = 5;

    // a = a / b
    a /= b;
    printf("%d", a);
    return 0;
}
```

Output

2

5. Modulus Assignment (%=)

Takes the modulus of the left operand by the right operand and stores the result in the left operand.

```
#include <stdio.h>
```

```
int main()
{
    int a = 10, b = 5;
```

```
// a = a % b  
a %= b;  
printf("%d", a);  
return 0;  
}
```

Output

0

6. Bitwise AND Assignment (&=)

Performs a bitwise AND operation and assigns the result.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
// 60 = 0011 1100 in binary
```

```
int a = 60;
```

```
// 13 = 0000 1101 in binary
```

```
int b = 13;
```

```
// Bitwise AND Assignment a = a & b -> 60 & 13 = 12 (0000 1100 in binary)
```

```
a &= b;
```

```
printf("%d", a);
```

```
return 0;
```

```
}
```

Output

12

7. Bitwise OR Assignment (|=)

Performs a bitwise OR operation and assigns the result.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // 60 = 0011 1100 in binary
```

```
    int a = 60;
```

```
    // 13 = 0000 1101 in binary
```

```
    int b = 13;
```

```
    // Bitwise OR Assignment a = a | b -> 60 | 13 = 61 (0011 1101 in binary)
```

```
    a |= b;
```

```
    printf("a |= b: %d\n", a);
```

```
    return 0;
```

```
}
```

Output

```
a |= b: 61
```

8. Bitwise XOR Assignment (^=)

Performs a bitwise XOR operation and assigns the result.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // 60 = 0011 1100 in binary
```

```
    int a = 60;
```

```

// 13 = 0000 1101 in binary
int b = 13;

// a = a ^ b -> 60 ^ 13 = 49 (0011 0001 in binary)
a ^= b;
printf("%d", a);
return 0;
}

```

9. Bitwise Left Shift Assignment (<<=)

Shifts the bits of the left operand to the left by the number of positions specified by the right operand and assigns the result.

```
#include <stdio.h>
```

```

int main()
{

```

```

// 60 = 0011 1100 in binary
int a = 60;

```

```

// 13 = 0000 1101 in binary
int b = 13;

```

```

// Bitwise Left Shift Assignment a = a << 2 -> 60 << 2 = 240 (1111 0000 in binary)
a <<= 2;
printf("%d", a);

```

```

return 0;
}

```

Output

240

10. Bitwise Right Shift Assignment (>>=)

Shifts the bits of the left operand to the right by the number of positions specified by the right operand and assigns the result.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // 60 = 0011 1100 in binary
```

```
    int a = 60;
```

```
    // 13 = 0000 1101 in binary
```

```
    int b = 13;
```

```
    // Bitwise Right Shift Assignment a = a >> 2 -> 60 >> 2 = 15 (0000 1111 in binary)
```

```
    a >>= 2;
```

```
    printf("%d", a);
```

```
    return 0;
```

```
}
```

Output

15

Conditional or Ternary Operator (?:) in C

The conditional operator (?:) is a shorthand way of writing simple [if-else statement](#). It evaluates a condition and returns one of two expressions based on whether the condition is true or false.

- Reduces code length by replacing simple if-else statements with a single expression.
- Improves code readability when performing straightforward conditional assignments.

```
#include <stdio.h>
```

```
int main() {
```

```
    int a = 10, b = 20;
```

```

int max = (a > b) ? a : b;

printf("Maximum = %d", max);

return 0;
}

```

Output

Maximum = 20

Syntax

The conditional operator can be in the form

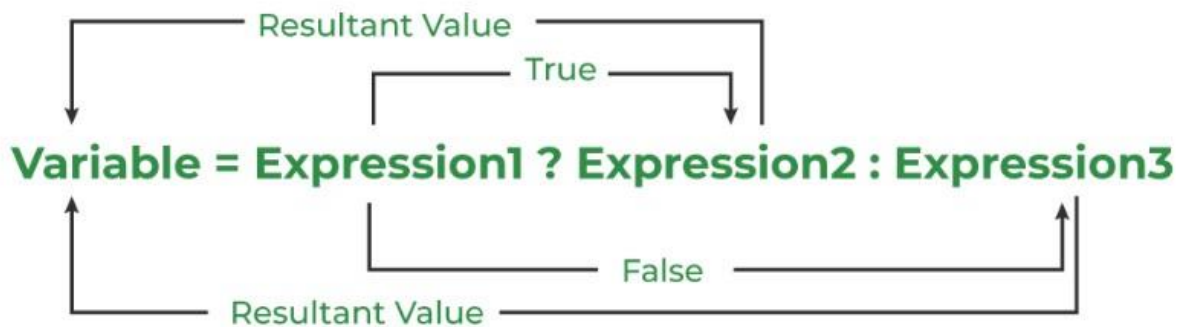
variable = Expression1 ? Expression2 : Expression3;

Or the syntax can also be in this form

variable = (condition) ? Expression2 : Expression3;

Or syntax can also be in this form

(condition) ? (variable = Expression2) : (variable = Expression3);



Conditional/Ternary Operator in C

It can be visualized into an if-else statement as:

```

if(Expression1){
variable = Expression2;
}else{
variable = Expression3;
}

```

Since the Conditional Operator '?' takes three operands to work, hence they are also called ternary operators.

Note: The conditional (?:) operator has one of the lowest operator precedences in C. Use parentheses when combining it with other operators to avoid unexpected results.

There are multiple examples some of them are:

Example 1: C Program to Store the greatest of the two Numbers using the ternary operator

```
// C program to find largest among two
```

```
// numbers using ternary operator
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int m = 5, n = 4;
```

```
    (m > n) ? printf("m is greater than n that is %d > %d",  
                    m, n)
```

```
        : printf("n is greater than m that is %d > %d",  
                    n, m);
```

```
    return 0;
```

```
}
```

Output

m is greater than n that is 5 > 4

Example 2: C Program to check whether a year is a leap year using ternary operator

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int yr = 1900;  
  
    (yr%4==0) ? (yr%100!=0? printf("The year %d is a leap year",yr)  
    : (yr%400==0 ? printf("The year %d is a leap year",yr)  
    : printf("The year %d is not a leap year",yr)))  
    : printf("The year %d is not a leap year",yr);  
    return 0;  
}
```

//This code is contributed by Susobhan AKhuli

Output

The year 1900 is not a leap year